ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Introduction to Python

Faculty of Information Technology

ĐẠI HỌC HOA SEN

Objectives

- ✓ Python—The Programming Language
- ✓ PyPI—The Python Package Index
- ✓ SciPy

The Python language

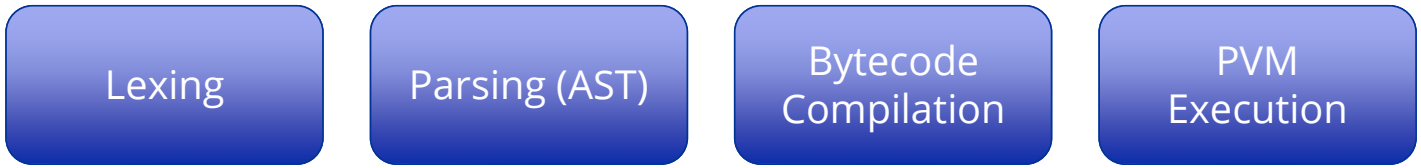
- Made by interpreters, tools, editors, libraries, notebooks, and so on.
- Interpreted programming language
  - Portable across operating systems
  - Object-oriented & multi-paradigm
  - Interfaced with C/C++/FORTRAN
  - Open-source & community-driven

- **Python** was created by **Guido van Rossum** in 1991, evolving from the ABC language. It is described as interpreted, portable, object-oriented, interactive, interfaced, open source, and easy to use.
- Python is an **interpreted language**, meaning it runs code through an interpreter rather than compiling it like C, C++, or Java. This makes development flexible and eliminates a separate compile step.
- It is **highly portable**, running on platforms such as Windows, Linux, and Mac without changing the code. This portability also makes it suitable for small devices like Raspberry Pi and microcontrollers.
- Python supports **object-oriented programming**, including classes and inheritance, while also allowing functional and vector-based approaches. It includes exception handling and offers flexible programming styles.

- It is **interactive**, allowing users to execute code line by line and immediately see results. This makes it popular in scientific computing, similar to environments like MATLAB.
- Python can be **interfaced with other languages** such as C, C++, and FORTRAN. This helps overcome its main weakness—slower execution speed—by integrating faster compiled code when needed.
- It is **open source**, with CPython as the main implementation, maintained by the Python Software Foundation. A large developer community continuously improves the language and its libraries.
- Finally, Python is known for its **simplicity and readability**, making it beginner-friendly while still powerful and widely used across many computing fields.

Execution Modes

- Script execution: python file.py
  - Interactive REPL mode (>>>)
  - Immediate feedback loop
  - Suitable for experimentation and learning



Python Execution Phases (Conceptual Flow)

- ***Lexing***, or *tokenization*, is the initial phase in which the Python (human-readable) code is converted into a sequence of logical entities, the so-called lexical tokens (see Figure 2-1).
- ***Parsing*** is the next stage in which the syntax and grammar of the lexical tokens are checked by a parser, which produces an abstract syntax tree (AST) as a result.

- ***Compiling*** is the phase in which the compiler reads the AST and, based on this information, generates the Python bytecode (.pyc or .pyo files), which contains very basic execution instructions. Although this is a compilation phase, the generated bytecode is still platform-independent, which is very similar to what happens in the Java language.
- The last phase is ***interpreting***, in which the generated bytecode is executed by a *Python virtual machine (PVM).*

Comparison of Python Implementations

Interpreter	Language Base	Key Feature	Performance Focus
CPython	C	Standard implementation	Moderate
Cython	C extension	Compile Python to C	High
Jython	Java	Runs on JVM	JVM dependent
IronPython	.NET	Integration with .NET	.NET dependent
PyPy	RPython	JIT compilation	High
RustPython	Rust	Rust-based interpreter	Experimental

PyPI and pip

- PyPI: Central Python package repository
  - Managed by global open-source community
  - pip: package manager tool
  - Install, update, remove packages
  - Dependency resolution support

Scientific Python Ecosystem (SciPy Stack)

- Alternative to MATLAB and R
  - Core libraries: NumPy, Pandas, matplotlib
  - Foundation for data science and AI
  - Strong academic and research adoption

NumPy – Numerical Computing Core

- Provides ndarray (N-dimensional array)
  - Vectorized element-wise operations
  - Broadcasting mechanisms
  - Foundation for Pandas & SciPy
  - High-performance numerical computing

Pandas – Data Analysis Framework

- DataFrame & Series structures
  - Label-based indexing
  - Data cleaning and transformation
  - Aggregation & group operations
  - Integration with SQL and CSV/Excel

matplotlib – Visualization Library

- 2D plotting framework
  - Line, bar, histogram, scatter plots
  - Scientific publication-quality graphs
  - Integration with NumPy & Pandas

Summary

- Python combines simplicity with power
  - Interpreter architecture enables flexibility
  - Multiple implementations optimize different goals
  - PyPI enables ecosystem scalability
  - SciPy stack powers modern data analysis


Q&A